

Game tech

bachelor in de elektronica – ICT – Brugge

docent: Van Gaever T.

academiejaar 2026-2027

Opdracht 2: IR – receiver

LaserTag Game

Infrarood Communicatie met STM32L432KC

Gebaseerd op ST Application Note AN4834

Hardware: 2x STM32 Nucleo-L432KC

Receivers: 3x TSOP4838 (38kHz)

Protocol: RC5 @ 38kHz carrier

1. Inleiding

In dit vak ga je een volledig functioneel LaserTag-systeem bouwen met behulp van infrarood communicatie. Je maakt gebruik van de STM32L432KC microcontroller en past het RC5-protocol toe, zoals beschreven in de ST Application Note AN4834.

Het RC5-protocol is oorspronkelijk ontwikkeld door Philips voor afstandsbedieningen en gebruikt Manchester-codering met een 36kHz draaggolf. In dit project wijken we af naar 38kHz om optimaal gebruik te maken van de beschikbare TSOP4838 IR-ontvangers.

2. Inleiding: IR receiver

In deze fase bouw je de IR-receiver die het RC5-signaal ontvangt.

2.1. TSOP4838 IR-ontvangmodule

De TSOP4838 is een geïntegreerde IR-ontvang- en demodulatormodule voor 38 kHz-gemoduleerde IR-signalen. De module filtert ruis en demoduleert het draaggolfsignaal, zodat alleen het envelope-signaal (de logische bitstructuur) op de uitgang verschijnt.

Parameter	Waarde / Beschrijving
Draaggolffrequentie	38 kHz (compatibel met RC5 36 kHz en SIRC 40 kHz)
Voedingsspanning	2.5 V tot 5.5 V (gebruik 3.3 V voor STM32)

Uitgang	Actief laag, open-collector equivalent
Signaalpolariteit	OMGEKEERD: idle = hoog, burst = laag
Aansluitvolgorde	Pin 1 = OUT, Pin 2 = GND, Pin 3 = VS (let op: in TO-92 behuizing)

Aansluitcircuit TSOP4838 (aanbevolen)

VS (pin 3) --> 100 ohm weerstand --> 3.3V

100 nF condensator van VS naar GND (zo dicht mogelijk bij de module)

GND (pin 2) --> GND

OUT (pin 1) --> GPIO input STM32 (bijv. PA0)

Tip: De 100 ohm + 100 nF vormen een RC-filter die ruis op de voeding onderdrukt.

2.2. RC5-protocol

Het RC5-protocol (Philips) is een 14-bits woord dat Manchester-codering gebruikt op een 36 kHz (38kHz in ons geval) draaggolf. Elk bit heeft een vaste lengte van 1.778 ms. Een logische '0' bestaat uit een burst in de eerste helft; een logische '1' uit een burst in de tweede helft van de bittijd.

Framestructuur (14 bits):

- Start bit (S): 1 bit, altijd '1'
- Field bit (F): 1 bit, selecteert bovenste of onderste commando-veld
- Toggle bit (C): 1 bit, wisselt bij elke toetsdruk (onderscheid herhaald signaal vs. nieuwe druk)
- Adres: 5 bits (32 apparaten mogelijk)
- Commando: 6 bits (64 commando's per apparaat)

Parameter	Waarde / Beschrijving
Halve bitperiode	889 us (min 640 us, max 1140 us)
Volledige bitperiode	1778 us (min 1340 us, max 2220 us)
Berichtlengte	24.889 ms (14 bits)
Herhalingstijd	113.778 ms (64 bitperioden pauze)
Draaggolffrequentie	36 kHz (38 kHz), duty cycle 25-33%

2.3. RC5 Decodering: Werking

De RC5-decodering maakt gebruik van TIM2 in PWM Input mode. In deze modus vangen twee kanalen van dezelfde timer elk afzonderlijk een flank op: kanaal 1 triggert op de neergaande flank, kanaal 2 op de stijgende flank. Zo meet de timer automatisch zowel de volledige pulsperiode als de duur van de lage puls.

Uit die twee metingen samen bepaalt de bibliotheek de waarde van elk bit. Manchester-codering betekent immers dat een bit niet door zijn absolute niveau bepaald wordt, maar door de overgangsrichting in het midden van de bittijd:

- Logische '1': overgang van laag naar hoog in het midden van de bittijd (lage puls in de tweede helft)
- Logische '0': overgang van hoog naar laag in het midden van de bittijd (lage puls in de eerste helft)

Omdat het TSOP4838-sigitaal geïnverteerd is (idle = hoog, burst = laag), zijn de logische niveaus omgedraaid ten opzichte van het originele RC5-sigitaal. De bibliotheek houdt hier automatisch rekening mee.

De timer genereert drie soorten events die de bibliotheek gebruikt:

- Neergaande flank (falling edge): meet de periode tussen twee opeenvolgende neergaande flanken. Dit geeft de totale pulslengte: T (889 us) of 2T (1778 us).
- Stijgende flank (rising edge): meet de duur van de lage puls (van neergaande naar stijgende flank). Samen met de totale periode bepaalt dit de bitwaarde.
- Timer overflow (update event): als er 3.7 ms geen flank gedetecteerd wordt, reset de bibliotheek het huidige pakket. Dit voorkomt dat een onvolledig frame als geldig beschouwd wordt.

Nadat alle 14 bits correct ontvangen zijn, zet de bibliotheek de globale vlag RC5FrameReceived op YES. De hoofdloop pollt deze vlag en roept RC5_Decode() aan om het frame op te splitsen in adres, commando en toggle bit.

3. Opdracht: IR Ontvanger Implementatie

3.1. Hardwareopstelling

Eerst geven we de hardware opstelling mee, het is belangrijk om dit te weten voor je gaat coderen.

Sluit alle componenten aan op het Nucleo-32 board zoals hieronder beschreven. Controleer de polariteit van alle componenten voor je de voeding aanzet.

Parameter	Waarde / Beschrijving
TSOP4838 OUT (pin 1)	PA0 -- als TIM2_CH1 input voor PWM-meting
TSOP4838 GND (pin 2)	GND
TSOP4838 VS (pin 3)	3.3V via 100 ohm + 100nF naar GND

UART TX (debugging)	PA2 -- intern verbonden met ST-Link VCP
UART RX (debugging)	PA15 -- intern verbonden met ST-Link VCP

Deze aansluitingen maak je pas na het instellen van de pinnen en opstellen van de codeeromgeving.

3.2. STM32CubeMX Projectconfiguratie

Ieder STM32 project begint met de correcte configuratie van de CubeMX omgeving.

We hebben het volgende nodig:

- Timer met interrupts die de flanken detecteert op PA0
- VCP (uart2) voor debugging via putty.

De correcte instellingen probeer je zelf te vinden, dit is zeer belangrijk en de start van het project.

3.3. UART Debugging via PuTTY

Parameter	Waarde / Beschrijving
USART2 TX	PA2 -- intern verbonden met ST-Link VCP ontvanger (RX)
USART2 RX	PA15 -- intern verbonden met ST-Link VCP zender (TX)
PC-verbinding	USB Micro-B kabel van PC naar CN1 (ST-Link USB connector op Nucleo)

3.3.1. USART2 Configuratie in STM32CubeMX

Stel in het .ioc bestand de USART2-parameters in als volgt:

- Mode: Asynchronous
- Baud Rate: 115200 Bits/s
- Word Length: 8 Bits (inclusief parity bit indien actief)
- Parity: None
- Stop Bits: 1
- Data Direction: Receive and Transmit
- Over Sampling: 16 Samples
- Hardware Flow Control: None (geen RTS/CTS)

Genereer de code. STM32CubeIDE maakt automatisch de functie `MX_USART2_UART_Init()` en de handler `huart2` aan.

3.3.2. `printf()` omleiden naar UART

Standaard stuurt de C-bibliotheek de uitvoer van `printf()` naar de syslog (semihosting) of nergens. Om `printf()` om te leiden naar USART2, moet je de low-level write-functie overschrijven. Voeg de volgende code toe aan je `main.c` (of een apart bestand `syscalls.c`):

```
#include "usart.h"

/* Overschrijf de low-level write-functie voor printf() */
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 100);
    return ch;
}
```

Zorg er ook voor dat de `stdio`-buffering uitgeschakeld is, zodat berichten direct zichtbaar zijn:

```
/* In main(), na MX_USART2_UART_Init() */
setvbuf(stdout, NULL, _IONBF, 0); /* Schakel buffering uit */
```

Als alternatief voor `__io_putchar()` kun je de `_write()` syscall overschrijven:

```
int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart2, (uint8_t *)ptr, len, HAL_MAX_DELAY);
    return len;
}
```

3.3.3. Debugberichten versturen

Voeg `printf()`-aanroepen toe na een succesvolle IR-decodering:

```
/* In de hoofdloop of in de RC5-callback */
if (RC5FrameReceived != RESET) {
    RC5_Decode(&IR_FRAME);
    printf("[RC5] Adres: 0x%02X | Commando: 0x%02X | Toggle: %d\r\n",
           IR_FRAME.Address,
           IR_FRAME.Command,
           IR_FRAME.ToggleBit);
    RC5FrameReceived = RESET;
}
```

3.4. De Decoderbibliotheek gebruiken met GitHub Copilot

In plaats van de bibliotheekfuncties manueel op te zoeken, gebruik je GitHub Copilot als AI-assistent om de integratiecode te genereren. De werkwijze is eenvoudig: je geeft Copilot de headerbestanden van de bibliotheek als context, en vraagt hem daarna gerichte code te schrijven. Copilot begrijpt de functiesignaturen, structuren en constanten uit de headers en schrijft correcte, werkende code op basis daarvan.

Belangrijk: Copilot genereert code op basis van wat jij hem geeft. Zonder de headerbestanden als context zal hij gokken en mogelijk verkeerde functienamen of parameters gebruiken. Voeg de headers altijd toe voordat je een vraag stelt.

Stap 1 – Open de Copilot Chat in STM32CubeIDE

STM32CubeIDE ondersteunt GitHub Copilot via de ingebouwde chatfunctie. Open het chatvenster via het menu Help > GitHub Copilot Chat, of gebruik de sneltoets Ctrl+Shift+I. Als Copilot nog niet geactiveerd is, log in met je GitHub-account (studenten krijgen Copilot gratis via GitHub Education).

Stap 2 – Voeg de headerbestanden toe als context

Open de bestanden rc5_decode.h en rc5_encode.h in de editor. Klik in het Copilot chatvenster op het paperclip-icoon ("Add context") en selecteer beide open bestanden, of typ # gevolgd door de bestandsnaam om ze als referentie toe te voegen. Copilot leest nu de functiedefinities, structuren en constanten rechtstreeks uit je project.

Voeg ook je main.c toe als context zodat Copilot weet welke variabelen en initialisaties al aanwezig zijn.

Stap 3 – Stel gerichte vragen

Stel concrete vragen met voldoende context. Enkele voorbeeldvragen die goed werken:

```
"Gebruik rc5_decode.h om in de hoofdlus een ontvangen RC5-frame te decoderen en het adres en commando via UART te printen."  
"Schrijf een GPIO interrupt callback voor PA6 die RC5_Encode_SendFrame() aanroept met debouncing van 200 ms, gebruik rc5_encode.h als referentie."  
"Leg uit wat RC5FrameReceived doet en hoe ik dit correct moet controleren in de while-lus."
```

Wees specifiek: vermeld de functienaam, de pin, het protocol en het gewenste gedrag. Hoe concreter de vraag, hoe bruikbaarder de gegenereerde code.

Stap 4 – Controleer en test de gegenereerde code

Copilot kan fouten maken. Controleer altijd of de gegenereerde code overeenkomt met de functiesignaturen in de headerbestanden. Compileer, flash en verifieer het gedrag via PuTTY of de oscilloscoop voordat je verdergaat. Als de code niet compileert, plak de foutmelding terug in de chat: Copilot lost ze meestal zelf op.

4. Oscilloscoopmeting: Envelope-sigitaal TSOP4838

De TSOP4838 demoduleert het ontvangen 38 kHz-sigitaal en presenteert het envelope-sigitaal (de logische bitstructuur) op zijn uitgang. Dit sigitaal is actief laag (omgekeerd): idle = hoog

(3.3V), burst ontvangen = laag (0V). Met een oscilloscoop visualiseer je de precieze timing van het IR-protocol.

4.1. Oscilloscoop Aansluiten

- Sluit de oscilloscoopmeetpen (probe) aan op de OUT-pin (pin 1) van de TSOP4838.
- Sluit de aardklem (crocodillenklem) van de probe aan op GND van het breadboard.
- Gebruik een 10x probe om de capacitieve belasting te minimaliseren.
- Stel de probe-compensatie correct in (vierkante golf compensatietest).

4.2. Oscilloscoop Instellen

1. Kanaalinstelling: stel Volt/div in op 1 V/div (signaal loopt van 0V tot 3.3V).
2. Tijdbasis: stel Time/div in op 1 ms/div voor RC5 (totale frame ~25 ms = ca. 25 divisies). Gebruik 500 us/div voor gedetailleerde bitmeting.
3. Koppeling: stel in op DC-koppeling (niet AC, want het signaal heeft een DC-component).
4. Trigger: stel in op neergaande flank (Falling Edge), niveau 1.5V (midden van het signaaltraject).
5. Trigger mode: stel in op 'Single' (enkelvoudige acquisitie) om een volledig frame op te vangen.
6. Richt de afstandsbediening op de TSOP4838 en druk een knop in. Het frame wordt eenmalig vastgelegd.
7. Gebruik 'Cursor'-functies of 'Measure' om tijdsintervallen nauwkeurig te meten.

4.3. Verwacht Signaalpatroon

RC5 protocol (afstandsbediening op de TSOP4838 gericht):

- Het signaal is idle hoog (3.3V) zonder IR-activiteit.
- Bij ontvangst van een RC5-frame zie je een reeks van lage pulsen. Elke 'lage' periode duurt 889 us (halve bitperiode) of 1778 us (volledige bitperiode) naargelang de bitwaarde.
- De startbit geeft altijd een neergaande flank na de idle-periode.
- Het volledige frame duurt circa 24.9 ms voor 14 bits.

SIRC protocol:

- Startbit: lage puls van 2.4 ms, gevolgd door hoge puls van 600 us.
- Bit '1': lage puls 1.2 ms + hoge puls 600 us (totaal 1.8 ms).
- Bit '0': lage puls 600 us + hoge puls 600 us (totaal 1.2 ms).

Meetopdrachten Oscilloscoop (verplicht te rapporteren in verslag)

1. Vang een volledig RC5-frame op. Meet de bitlengte van drie willekeurige bits.
Vergelijk de gemeten waarden met de theoretische 1778 us. Noteer de afwijking.
2. Identificeer de startbit, field bit en toggle bit in het frame.
3. Veranderd de toggle bit? Kan je dit aanduiden? Wat is het nut hiervan?
5. Sla een schermafdruck op (print screen of foto) voor gebruik in het verslag.

5. Verslag

Dien een verslag in met de volgende onderdelen:

- Schematisch overzicht van de volledige hardware-opstelling (zender + ontvanger, inclusief alle componenten en pinverbindingen)
- Schermafdruckken van PuTTY met minimaal vijf ontvangen IR-frames van verschillende knoppen
- Oscilloscoopmeting van het envelope-signaal (TSOP4838): bewaar de afbeelding en annotateer de bitperioden.
- Bespreking van ondervonden problemen en toegepaste oplossingen